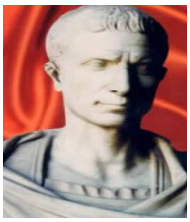
Resuelve los siguientes ejercicios y entrega el código fuente java tanto en un fichero .pdf como en ficheros de texto, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean, si las hay.

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1. JULIO CÉSAR Y LA INFORMÁTICA.



El cifrado de césar lo usaba Julio César en el siglo I a.d.C. El algoritmo original consiste en cambiar cada letra del mensaje por la letra que está 3 posiciones a la derecha en el alfabeto (las letras A por letras D, las B por E, ..., las X por A, las Y por B y las Z por C).

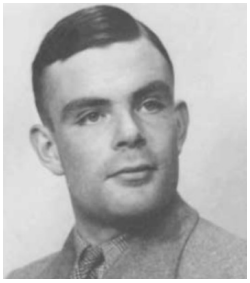
Es **un cifrador de sustitución monoalfabético** (una letra se cambia por otra, siempre el mismo cambio) **en el que las operaciones se realizan módulo n**, siendo n el número de letras del alfabeto (originalmente el latín tenía 26 letras). **Matemáticamente**: cada letra del alfabeto se codifica en un número (según su posición en el alfabeto) empezando por 0 la primera letra, 1 la siguiente, … n la última. Las operaciones a realizar:

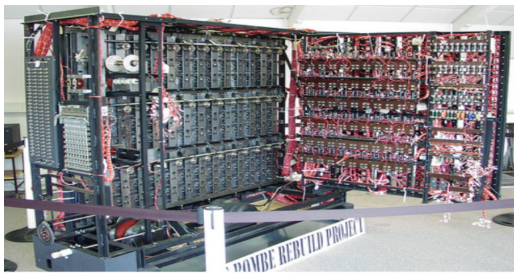
Encriptar mensaje de m letras (M = a1 a2.... an) con clave K (1 <= k < módulo)
cifrado ← ""
Para cada letra ai de M que sea del alfabeto hacer
cifrado = cifrado + letra de alfabeto en la posición (ai + K) mod tamaAlfabeto

Desencriptar mensaje de m letras (C = c1 c2 … cm) con clave K:
mensaje ← ""
Para cada letra ci de C que sea del alfabeto hacer
letra ← (cifra – 3) mod tamaAlfabeto

A pesar de su simplicidad, el cifrado césar fue utilizado por los oficiales sudistas en la Guerra de Secesión americana y por el ejército ruso en 1915 durante la primera guerra mundial. Hasta este conflicto, la criptografía se caracterizó en que no importa como de astutos fueran los criptógrafos de uno u otro bando, los criptoanalistas del otro rompían los criptosistemas.

En la segunda guerra mundial los nazis perfeccionaron esta situación, utilizando una máquina llamada **ENIGMA** que tenía tantas claves posibles, que parecía imposible de romper. **Alan Turing** participó en su criptoanálisis, y además de acortar la guerra unos años, provocó el nacimiento de la informática, ya que el primer ordenador digital que se construyó fue para desencriptar los mensajes de ENIGMA.

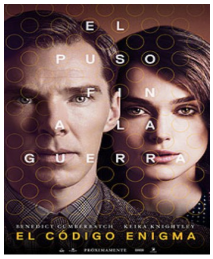

(a) Alan Turing.


(b) Colosus, el 1er ordenador.

(c) Enigma.

(d) In memoriam.

En 1954 después de un oscuro incidente con la justicia inglesa, Alan Turing se sucidó mordiendo una manzana envenenada con cianuro a modo de Blancanieves. Película (por si eres cinéfilo): **The Imitation Game** (descifrando enigma), 2014 Oscar: Mejor guión adaptado y 8 nominaciones más.



En este ejercicio, con el fin de practicar bucles y String, vas a implementar en el programa **U3EP01.java** un criptosistema clásico denominado el método de César, programa del que ves estas ejecuciones. Te aconsejo que no uses arrays, solo String, es más sencillo:

```
     CRIPTOGRAFIA CESAR

1. Configurar.
2. Encriptar.
3. Desencriptar.
4. Salir.

Escoja una opción (1-4):
```

```
     CRIPTOGRAFIA CESAR-CONFIGURAR

Alfabeto actual: ABCDEFGHIJKLMNOPQRSTUVWXYZ
Módulo actual:   26

  Nuevo alfabeto (enter para no cambiar):
  AABCDEFGHIJKLMNOPQRSTUVWXYZ 0123456789
    Tiene letras repetidas.
  Nuevo alfabeto (enter para no cambiar):
  ABCDEFGHIJKLMNOPQRSTUVWXYZ 0123456789
  Módulo actual:   37

    Pulse ENTER para volver al menú
```

```
     CRIPTOGRAFIA CESAR-ENCRIPTAR

Clave (1-36): 0
Clave (1-36): 2
Texto en claro: HOLA CARACOLA_
 Texto Filtrado: HOLA CARACOLA
 Texto cifrado:  JQNC1ECTCEQNC
    Pulse ENTER para volver al menú
```

```
     CRIPTOGRAFIA CESAR-DESENCRIPTAR

Clave (1-36): 37
Clave (1-36): 2
Texto cifrado:    JQNC1ECTCEQNC
  Cifra filtrada: JQNC1ECTCEQNC
  Texto en claro: HOLA CARACOLA
    Pulse ENTER para volver al menú
```

Debes tener en cuenta que:

- **El alfabeto:** No puede tener letras repetidas. Se usa para eliminar las letras incorrectas (que no sean del alfabeto) de los mensajes y las cifras antes de encriptar/desencriptar. La cantidad de letras define el valor del módulo a usar. Al cambiar el alfabeto, cambias el resultado de las operaciones.
- **El módulo N:** Se utiliza para realizar la encriptación y desencriptación convirtiendo el resultado de las operaciones en circular (nunca sales de los valores 0, 1, …, N-1). Ojo a la hora de desencriptar que el operador módulo de los lenguajes de programación no se comporta igual que en matemáticas. Por ejemplo, en matemáticas la operación –3 % 5 = 2 pero en Java y en cualquier lenguaje -3 % 5 = -3. Debes conseguir el comportamiento matemático (pero es sencillo de hacer, si el resultado es negativo…).
- **La clave K es el desplazamiento**. Si usas una clave k, distinta de 3 obtienes otra cifra. El criptosistema tiene tantas claves posibles como letras el alfabeto (pero las claves 0 y N (tamaño del alfabeto) no son útiles porque la cifra se queda como el mensaje en claro).

EJERCICIO 2. TRABAJAR CON ARRAYS DE 1 DIMENSIÓN.

Crea un programa en Java llamado **U3EP02.java** que defina un array de 12 Strings y lo inicialice con los nombres de los doce meses del año. Luego define otro array de 12 doubles que va a almacenar los mm de precipitaciones de cada mes de un lugar. Este array se rellena con valores pedidos por teclado, tal y como se ve en la imagen.

Luego. El programa muestra por consola:

- El nombre del primer mes más lluvioso.
- La media de lluvias
- La mediana
- La desviación típica

```
        PRECIPITACIONES ANUALES
        =======================

enero: 1
febrero: 3
Marzo: 0
Abril: 3
Mayo: 2
Junio: 4
julio:  5
Agosto: 2
Septiembre: 24
Octubre: 4
Noviembre: 6
diciembre: 10

El mes más lluvioso septiembre con 24.0 mm
Media: 5.25 mm
Mediana: 7.0 mm
Desviación: 6.033 mm
```

CALCULAR LA MEDIA:

MEDIA

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

CALCULAR LA MEDIANA

Ordenar datos de menor a mayor
Si el nº de datos es impar. El dato central.
Si el nº de datos es par, la media de los 2 datos centrales.

CALCULAR LA DESVIACIÓN TÍPICA

$$\sigma = \sqrt{\frac{\sum_1^N (x_i - \bar{X})^2}{N}}$$

- **X** → Variable
- **N** → Número de observaciones.
- **X_i** → Observación número i de la variable X.
- **$\bar{X}$** → Es la media de la variable X.

EJERCICIO 3. TRABAJAR CON ARRAYS DE 2 DIMENSIONES.

Un alumno de informática quiere registrar las horas de estudio mensuales dedicadas a cada una de sus asignaturas. Crea un programa llamado **U3EP03.java** que declare un array de strings llamado asignaturas y una tabla de doubles llamada horas y muestre el siguiente menú:

```
            HORAS DE ESTUDIO
            ================

1. Definir tablas.
2. Modificar nota.
3. Total horas por asignatura.
4. Total horas por meses.
5. Nombre y horas de asignatura más estudiada.
6. Salir.

   Escoja una opción (1-6):
```

Según la opción escogida por el usuario:

- **Si elige la opción 1:** pregunta por cuantas asignaturas quiere tener en las tabla. Luego crea el array de asignaturas (una dimensión) y pregunta y almacena los nombres. Luego crea el array de notas con tantas filas como asignaturas y tantas columnas como meses tiene el año.

```
   Escoja una opción (1-6): 1

Cuántas asignaturas: 5
  Asignatura 1: GBD
  Asignatura 2: LMSGI
  Asignatura 3: FOL
  Asignatura 4: INGLES
  Asignatura 5: PROG

  Pulser ENTER para menú...
```

	Enero	Febrero	...	Diciembre
Asignatura 1				
...				
Asignatura 5				
TOTAL				

- **Si elige la opción 2**, pregunta por el nombre de una asignatura. Localiza su fila buscándola en el array de asignaturas. Si no la encuentra da un aviso y vuelve al menú, si la encuentra, pregunta por un número de mes repetidamente hasta que sea correcto (1-12) y pregunta por las horas que almacena en la celda adecuada del array horas.

```
   Escoja una opción (1-6): 2

Asignatura: LM
    No encontrada, pulse ENTER para menú...
```

```
   Escoja una opción (1-6): 2

Asignatura: GBD
Mes (1-12): 0
Mes (1-12): 1
Horas: 20

  Pulser ENTER para menú...
```

- **Si elige la opción 3**, calcula el total anual de horas dedicadas a cada asignatura y muestra la salida formateada de forma que el nombre de la asignatura tenga un ancho mínimo de 10 y el de las horas 6 con 2 decimales.

```
   Escoja una opción (1-6): 3

      HORAS ANUALES POR ASIGNATURA
GBD         20.00
LMSGI        0.00
FOL          0.00
INGLES       0.00
PROG         0.00

  Pulser ENTER para menú...
```

- **Si elige la opción 4** calcula y muestra el total mensual de horas dedicadas a estudiar.

```
   Escoja una opción (1-6): 4

      HORAS TOTALES MENSUALES
 1     20.00
 2      0.00
 :        :
11      0.00
12      0.00

  Pulse ENTER para menú...
```

- **Si elige la opción 5** muestra el nombre y el total de horas de la asignatura más estudiada durante todo el año.

```
  Escoja una opción (1-6): 5

La más estudiada GBD con 20.00 horas

 Pulse ENTER para menú...
```

EJERCICIO 4. TRABAJAR CON ARRAYS ASIMÉTRICOS.

Crea el programa **U3EP04.java** que permita gestionar una agenda de contactos telefónicos. El programa se crea con el objetivo de no consumir inicialmente más memoria de la necesaria (aunque tampoco bajaremos tamaño de arrays, permitimos huecos). Utiliza los siguientes arrays:

- **contactos**: Un array de Strings donde almacena los nombres de los contactos.
- **direcciones**: Un array de arrays de Strings donde guardar números de teléfono, mails o direcciones postales de un contacto. Las direcciones del contacto 3 estarán almacenadas también en la posición 3 del array de direcciones, de forma que el programa debe mantener sincronizadas estas posiciones.

El programa declara los dos arrays, muestra el siguiente menú y lee una opción repetidamente hasta que sea correcta:

```
    AGENDA DE CONTACTOS
    ===================

1. Nuevo contacto.
2. Borrar contacto.
3. Modifica direcciones de un contacto.
4. Busca contactos
5. Salir.

Escoge una opción (1-5): 1
```

a) Opción 1. Si los arrays no existen aún, los crea de tamaño 1. Si ya existen, debes comprobar si hay huecos (posiciones a null) debidos al borrado anterior de contactos. Si no tienen huecos, debes crear nuevos arrays de una longitud 1 mayor y copiar los originales para finalmente sustituirlos. Una vez se asegura que hay hueco para el nuevo contacto, pregunta su nombre y lo almacena en la posición adecuada indicando la posición donde queda. Ejemplos:

```
Escoge una opción (1-5): 1

Nombre: Manolito
        Posición 0

  Pulse ENTER para menú...
```

```
Escoge una opción (1-5): 1

Nombre: Miguel
        Posición 1

  Pulse ENTER para menú...
```

```
Escoge una opción (1-5): 1

Nombre: Tania
        Posición 2

  Pulse ENTER para menú...
```

b) Opción 2. Te preguntará repetidamente la posición del contacto a borrar hasta que

sea correcta. El programa debe indicar las correctas (en verde en la figura). Cuando sea correcta, debe confirmar la acción, pues no puede deshacerse. Si el usuario confirma, pone a null las posición en los arrays de contactos y direcciones.

```
Escoge una opción (1-5): 2

Posición a borrar [0-2]: 3
Posición a borrar [0-2]: 0
  Quieres borrar el contacto Manolito (s/n)? s

     Contacto y direcciones borradas.

  Pulse ENTER para menú...
```

c) **La opción 3**. Si añades una dirección, procedes como al añadir contactos (ver si está creado, crearlo si no, ver si hay huecos, aumentarlo de tamaño sino) pero ahora en el array de direcciones de un contacto. Si borras una dirección, debes indicar qué posición borras y si es correcta, la dejas a null.

```
Escoge una opción (1-5): 3

Posición a Modificar [0-1]: 0
    No existe el contacto.
Posición a modificar [0-2]: 20
Posición a Modificar [0-2]: 1

    DIRECCIONES DE Miguel
    (I) Insertar  (B) Borrar  (S) Salir   Operación: I
    Insertar: 12345678

    DIRECCIONES DE Miguel
    0   12345678
    (I) Insertar  (B) Borrar  (S) Salir   Operación: I
    Insertar: migue@gmail.com

    DIRECCIONES DE Miguel
    0   12345678
    1   migue@gmail.com
    (I) Insertar  (B) Borrar  (S) Salir   Operación: B
    Borrar [0-1]: 0

    DIRECCIONES DE Miguel
    1   migue@gmail.com
    (I) Insertar  (B) Borrar  (S) Salir   Operación: B
    Borrar [0-1]: 0

    DIRECCIONES DE Miguel
    1   migue@gmail.com
    (I) Insertar  (B) Borrar  (S) Salir   Operación: I
    Insertar: 12345678

    DIRECCIONES DE Miguel
    0   12345678
    1   migue@gmail.com
    (I) Insertar  (B) Borrar  (S) Salir   Operación: S
```

d) **La opción 4**. Da dos opciones de búsqueda. Si una de ellas queda vacía, no se aplica. Si contiene alguna letra en la primera letra solo aparecen contactos cuyo nombre

comience por la frase indicada. Si la segunda frase no es vacía significa que solo aparecen los contactos cuyo nombre contiene la frase indicada. Mira los ejemplos:

```
Escoge una opción (1-5): 4

Comienza por la frase:
Contiene la frase:

    (1) DIRECCIONES DE Miguel
    0   12345678
    1   migue@gmail.com

    (2) DIRECCIONES DE Tania

    Pulse ENTER para menú...
```

```
Escoge una opción (1-5): 4

Comienza por la frase: Ta
Contiene la frase:

    (2) DIRECCIONES DE Tania

    Pulse ENTER para menú...
```

```
Escoge una opción (1-5): 4

Comienza por la frase:
Contiene la frase:    ig

   (1) DIRECCIONES DE Miguel
    0   12345678
    1   migue@gmail.com

    Pulse ENTER para menú...
```

EJERCICIO 5. MEDIR EL PARECIDO DE DOS TEXTOS.

A veces se necesita conocer lo parecidos o diferentes que son dos textos. Este trabajo lo suelen hacer los motores de búsqueda (yahoo, google, …) o herramientas de análisis de plagios, o de detección de emociones, etc. Una de las formas de hacerlo es calcular un índice entre [0-1] de forma que si los dos textos tienen un índice de coincidencia de 0 es que no pueden ser más diferentes y si tienen un índice de 1 es que son completamente iguales. Utiliza el siguiente método para calcular el índice de los textos t1 y t2:

- Se llama a al nº de palabras comunes de ambos textos.
- Se llama b al nº de palabras de t1 que no están en t2.
- Se llama c al nº de palabras de t2 que no están en t1.

$$\text{parecido} = \frac{|\text{t1 intersección t2}|}{|\text{t1 unión t2}|} = \frac{a}{a+b+c}$$

Ejemplo de ejecución:

```
C:\> java U3EP05.java
Texto1: el zumo de naranja es un clásico
Texto2: la naranja mecánica es un clásico del cine
  a = 4 b = 3 c = 4
  Parecido [0,1]: 0.36363636
```

EJERCICIOS JEDI (EL QUE TE TOQUE)

Como siempre, aunque solo tenéis que entregar el que os toca, deberíais intentar hacerlos todos para practicar.

EJERCICIO 6. BÚSQUEDAS SECUENCIALES Y BINARIAS.

Haz un programa que pruebe si conviene ordenar un array que se va a utilizar para hacer búsquedas. El programa define un array de 1 millón de enteros y lo rellena con números aleatorios entre 0 y 50000.

Ahora toma el tiempo que tarda en generar 1000 números aleatorios entre 0 y 100000, buscarlos de forma secuencial y mostrar el porcentaje de los que ha encontrado y los que no. Hazlo lo más eficiente posible.

Ahora, ordena el array de enteros y vuelve a repetir lo anterior pero las búsquedas las haces usando el algoritmo de búsqueda binaria (Mira la ejecución de ejemplo).

Nota: tus resultados no tienen porqué ser iguales a los míos.

```
C:\> java U3EP06.java
Generando y buscanco números de forma secuencial...
  Encontrados:    48.76%
  No encontrados: 51.24%
  Tiempo: 3.12 segundos

Generando, ordenando y buscanco números de forma binaria...
  Tiempo en ordenar:  0.60 segundos
  Encontrados:    49.13%
  No encontrados: 50.87%
  Tiempo: 0.42 segundos
```

EJERCICIO 7. MATRICES CASI NULAS.

Una matriz "casi nula" es una matriz con un alto porcentaje de elementos nulos. Una matriz "casi nula" con k elementos no nulos se suele representar almacenando los elementos no nulos en una matriz de k filas y 3 columnas, conteniendo cada columna de esta matriz la fila, la columna y el valor de los elementos no nulos, respectivamente. Por ejemplo:

$$\text{la matriz “casi nula”} \begin{pmatrix} 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \text{ se puede representar por } \begin{pmatrix} 1 & 3 & 3 \\ 3 & 2 & 6 \\ 3 & 3 & 1 \\ 5 & 5 & 1 \end{pmatrix}$$

a) (0.8 PUNTOS) Crea el programa **U3EP07.java** que pregunte por un número entero n y cree y defina una matriz de enteros de tamaño n x n y la rellene cada celda con números enteros entre 0 y 9 con probabilidad 0.1 o la deje a 0 con probabilidad 0.99. Tras rellenar mínimamente la tabla, la imprime y luego genera su representación en una matriz óptima que también imprime.

```
C:\> java U3EP07.java
Tamaño: 4
Matriz casi nula:
  | 0 0 3 0 |
  | 0 0 0 0 |
  | 0 0 0 0 |
  | 0 1 0 0 |

Matriz óptima:
  | 1, 3, 3 |
  | 4, 2, 1 |
```

b) (0.2 PUNTOS) Piensa como podrías hacer algo similar (representar una tabla casi nula de forma más eficiente desde el punto de vista del consumo de memoria), pero si en vez de tener enteros

como tipo base de la matriz casi vacía, tuvieses fechas (objetos de tipo Date).

EJERCICIO 8. CUADRADOS MÁGICOS.

Crea un programa llamado **U3EP08.java** que genere cuadrados mágicos. El programa lee un numero natural (2 < n < 11) y calcula una matriz mágica de orden n. Una matriz de orden n (tamano n × n) se dice que es mágica si contiene los valores 1, 2, 3, ..., n × n y cumple la condición de que la suma de los valores almacenados en cada fila y columna coinciden. Ejemplo:

8	1	6	**15**
3	5	7	**15**
4	9	2	**15**
15	**15**	**15**	

Para la construccion de la matriz mágica de orden impar se deben seguir la siguientes reglas:

- Colocamos el numero 1 en la celda correspondiente al centro de la primera fila.
- Seguimos colocando los siguientes números avanzando una celda hacia arriba y a la derecha. Consideramos que la matriz cumple la propiedad de la circularidad, es decir, si salimos por la derecha se vuelve a entrar por la izquierda, y si se sale por arriba se entra por abajo.
- Si la celda donde corresponde el siguiente numero de la lista esta ya ocupada, entonces se coloca este en la celda que haya debajo del ultimo numero colocado.

Para la construccion de la matriz mágica de orden par se deben seguir la siguientes reglas:

- Colocamos los numeros 1, 2, 3, ..., n × n consecutivamente, de izquierda a derecha y de arriba a abajo.
- Invertimos los valores de la diagonal principal (recorre las esquinas superior izquierda hasta inferior derecha).
- Invertimos los valores de la segunda diagonal principal (recorre las esquinas superior derecha hasta la esquina inferior izquierda).

16	2	3	13	**34**
5	11	10	8	**34**
9	7	6	12	**34**
4	14	15	1	**34**
34	**34**	**34**	**34**	

EJERCICIO 9. DETECTAR ELEMENTOS REPETIDOS DE UN ARRAY .

Haz el programa **U3EP09.java** que defina un array de n enteros llamado original y lo rellene con valores aleatorios de entre 0 y n. Crea otro array llamado repetidos con los valores repetidos y otro llamado diferentes con los valores que no estén repetidos.

```
C:\> java U3EP09.java
Teclee n: 5
  Original = {2, 4, 5, 2, 0}
  Repetidos = {2}
  Diferentes = { 4, 5, 0 }
```

Penalizaciones:
Si copias el array original: -20%
Si haces cambios en el array original: -20%

EJERCICIO 10. CODIFICAR/DECODIFICAR TEXTO EN NÚMEROS.

Haz el programa **U3EP10.java** que de 4 opciones (definir alfabeto, codificar, decodificar y salir).

a) Si se escoge la opción 1, pregunta por un alfabeto hasta que compruebe que no tiene letras repetidas. Usando este alfabeto como un sistema de numeración, su base será el número de letras que tenga. Usa el alfabeto para expresar mensajes con sus letras en un número decimal. Será similar a cuando pasas de binario a decimal, de octal a decimal, de hexadecimal a decimal, etc.

Pregunta por la cantidad de bits que pueden tener como máximo esos números y averigua cuántas letras de tu alfabeto pueden formar un mensaje para que quepa en esa cantidad de bits. Imprime esta información como tamaño de bloque y la almacenas en una variable.

```
C:\> java U3EP10.java
 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 7
 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 1

Alfabeto (sin repeticiones): AEIOOU
Alfabeto (sin repeticiones): AEIOU
Máximo de bits: 7
  El máximo valor es 127 y tamaño de bloque 3 letras

 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion:
```

b) Si escoge la opción 2, pregunta por un mensaje al usuario, elimina las letras que no sean del alfabeto y lo codificas en bloques del tamaño calculado. Si un bloque no alcanza el nº de letras calculado, rellena con ceros por la izquierda hasta que lo haga con la primera letra del alfabeto. Ejemplos:

```
C:\> java U3EP10.java
 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 2

Mensaje:  HOLA AMIGOS
Mensaje filtrado:  OAAIO
Bloques
  OAA   75
  AIO   13
```

c) Si escoge la opción 3, pregunta por un valor (un número decimal) y lo expresa como el mensaje que representa (lo decodifica). Ejemplos:

```
C:\> java U3EP10.java
 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 3

Valor (0-127):  128
Valor (0-127):  75
  Bloque:  OAA
```

```
C:\> java U3EP10.java
 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 1

Alfabeto  (sin repeticiones): 01
Máximo de bits: 7
  El máximo valor es 127 y tamaño de bloque 7 letras

 (1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 2
Mensaje:  100510001
Mensaje filtrado:  10010001
Bloques
  1001000   72
  0000001   1

(1)Alfabeto  (2)Codificar (3)Decodificar (4)Salir  Opcion: 3
Valor (0-127):  72
  Bloque:  1001000
```

EJERCICIOS OPCIONALES (+4 PUNTOS)

MÁS CRIPTOGRAFÍA CLÁSICA.

La criptografía es una ciencia (se nutre de matemáticas, informática y física) que se encarga de estudiar como proteger el almacenamiento e intercambio de información para conseguir su confidencialidad, detectar modificaciones no autorizadas, asegurar autoría (firmas y certificados digitales), impedir el repudio (que algo niegue su participación de forma falsa)...

Es más vieja que la picor (existe desde que la información se escribe, es importante y hay que exponerla a peligros). Al principio se usaba para encriptar (ocultar un mensaje) y desencriptar (recuperar el mensaje encriptado) y quedaba restringida a ambientes militares y políticos, embajadas, etc. Hoy en día su uso es cotidiano: un cajero, compras on-line, firmar digitalmente mensajes, generar claves seguras de sesión, criptomonedas, protocolos de autenticación, códigos hash, tunelizar comunicaciones en protocolos como VPN, SSH, HTTPS, encriptar mensajes en redes wifi, etc.

Un criptosistema en **un algoritmo A** y **una clave k** que transforma **mensajes M** de un alfabeto a **mensajes C** del mismo o de otro alfabeto. Las operaciones básicas que se necesitan son:

Encriptar(A, k, M) = Cak y **Desencriptar(A, k, Cak) = M**

Dos grandes maneras de hacer estas transformaciones (tipos de algoritmos A):

- **Algoritmos de Sustitución**: cambian cada letra del mensaje por otra. Si lo aplicamos solo a texto, es mejor definir un String con el alfabeto de nuestros mensajes y codificarlos (traducirlos) al alfabeto que use el criptosistema (puede ser diferente). Si usa números por ejemplo, los nºs se pasan al algoritmo del criptosistema junto con una clave (la clave puede ser un nº, una frase, varios nºs.). Si el criptosistema da otros nºs. Como resultado, se vuelven a codificar en letras del alfabeto original. De esta forma, el usuario solo interactúa con mensajes de texto (se pueden teclear, mostrar, etc.).

- **Algoritmos de Transposición**: cambian la posición que ocupan las letras en el mensaje, sin cambiar cada letra (solo desordenan las letras). En este caso no es necesario codificar/decodificar porque el alfabeto no cambia. Pero ahora igual surge la necesidad de rellenar mensajes para que tengan cierta longitud fija.

(+1 PUNTO) CRIPTOSISTEMA DE SUSTITUCIÓN: MONOALFABÉTICO GENERAL.

Al usar el método de César en un alfabeto de 26 letras, con una clave K (entre 0 y 25), lo que estamos haciendo es aplicar una permutación a las letras que consiste en asociar a cada letra, la que está k lugares a la derecha (en módulo 25) en el alfabeto y haciéndolo circular.

Pero en este alfabeto hay realmente unas 25! permutaciones posibles (25 x 24 x … 3 x 2) y con César (cambiando la clave k) solo podemos usar 25 de ellas (desplazamientos fijos).

Una forma de aumentar el espacio de claves (la cantidad de claves), es cambiar la forma de hacer las permutaciones. Suponiendo que usamos el alfabeto de las letras mayúsculas, elegimos una clave (ahora las claves son frases) por ejemplo: "STAR WARS". Quitamos las letras que no estén en el alfabeto y las letras repetidas. Como tiene las letras S, A y R repetidas, solamente se queda con STARW (quitamos letras repetidas en la clave, la frase sirve para fabricar la clave, como una semilla).

Ahora vamos escribiendo las letras no repetidas de la clave en el orden en que aparecen y más adelante, las que falten del alfabeto en el orden del alfabeto (pero sin repetir):

permutacion ← "STARWBCDEFGHIJKLMNOPQUVXYZ"

Emparejando esta cadena con la cadena del alfabeto se define la permutación que aplicamos al encriptar/desencriptar cada letra del alfabeto (la A del alfabeto (posición 0) por la letra S (posición 0 de la permutación), la B por T, la C por A, la D por R, la E por W, .. la Z por Z).

Letras alfabeto: ABCDEFGHIJKLMNOPQRSTUVWXYZ
Cifra permutación: STARWBCDEFGHIJKLMNOPQUVXYZ

EJEMPLO: HOLA → DLHS

A la hora de implementarlo necesitas 3 algoritmos: uno para generar la permutación a través de una contraseña, otro para encriptar y otro para desencriptar. No hay que codificar ni decodificar porque el criptosistema (la permutación) usa los mismos símbolos que el alfabeto. Haz el programa **U3PEX1.java** que presente un menú similar al del ejercicio 1 y permita definir el alfabeto, encriptar y desencriptar. Ejemplo:

```
  Elija una opción (1-4): 2

Mensaje: LOS PASTORES
   Eliminando letras fuera del alfabeto...
Clave (una frase): NAVIDAD JUBILOSA
   Generando clave...
   Alfabeto: ABCDEFGHIJKLMNOPQRSTUVWXY Z
   Permuta:  NAVID JUBLOSCEFGHKMPQRTWXYZ

Cifra: SFMYGNMPFKDM
```

```
  Elija una opción (1-4): 3

Cifra: SFMYGNMPFKDM
   Eliminando letras fuera del alfabeto...
Clave (una frase): NAVIDAD JUBILOSA
   Generando clave...
   Alfabeto: ABCDEFGHIJKLMNOPQRSTUVWXY Z
   Permuta:  NAVID JUBLOSCEFGHKMPQRTWXYZ

Mensaje: LOS PASTORES
```

(+1 PUNTO) UN CRIPTOSISTEMA DE TRANSPOSICIÓN: ESCÍTALO.

El escítalo era usado en el **siglo V a.d.C.** por el pueblo griego de los Lacedonios. Consiste en un bastón en el que se enrolla una cinta de cuero y luego se escribe en ella el mensaje de forma longitudinal. Al desenrollar la cinta de cuero, las letras aparecerán desordenadas ocultando el mensaje original.

Para descifrar el criptograma y recuperar el mensaje en claro habrá que volver a enrollar la cinta de cuero en un bastón con el mismo diámetro y longitud que el usado por el emisor y leer el mensaje de forma longitudinal. La clave del criptosistema es **la longitud y el diámetro** del bastón (dos números). Se trata de una cifra por transposición pues los caracteres del criptograma son los mismos que los del texto en claro, pero están distribuidos de otra forma. No se han sustituido unas letras por otras, solo se han desordenado.

Si tenemos un escítalo en el que podemos escribir 3 líneas de texto (diámetro) de hasta 9 caracteres cada una (longitud), matemáticamente equivale a escribir cada letra en una casilla de una tabla de 3 filas x 9 columnas. Una vez rellenas las casillas de la tabla, el mensaje cifrado sería el que se obtiene al copiar las letras ordenadas por columnas. Ejemplo:

Texto llano (escrito por filas): **ejemplodelmetododelescitalo**

e	j	e	m	p	l	o	d	e
l	m	e	t	o	d	o	d	e
l	e	s	c	i	t	a	l	o

Texto cifrado (leyendo por columnas): **elljmeeesmtcpoildtooaddleeo**

Para implementarlo, tendrás que dividir el mensaje en bloques. Por ejemplo, si el mensaje a encriptar tiene 100 letras y estamos usando 3x9 de claves, habrá que hacer bloques de 27 letras. Hacemos el primer bloque y encriptar, el segundo bloque y encriptar, el tercer bloque

y encriptar. Nos quedan 19 letras del mensaje, así que las ponemos en el último bloque y rellenamos (por ejemplo con espacios en blanco) lo que falte de la tabla.

Al ser un método de transposición, no hay que codificar/decodificar. Te aconsejo que uses un array de dos dimensiones de caracteres. Haz el programa **U3EPX2.java** de modo similar al ejercicio 1 de la práctica. Ejemplo:

```
    ESCITALO
    ========

1. Configurar clave.
2. Encriptar.
3. Desencriptar.
4. Salir.

Escoja una opción (1-4):
```

```
Escoja una opción (1-4): 1

 Filas: 3
 Columnas: 3
```

```
Escoja una opción (1-4): 2

Mensaje: Esto es una prueba
Cifra: Eoss  teunpearb ua
```

```
Escoja una opción (1-4): 3

Cifra: Eoss  teunpearb ua
Cifra: Esto es una prueba
```

(+2 PUNTOS) CIFRADOR POLIALFABÉTICO DE VIGENERE.

Hasta ahora los criptosistemas de sustitución que hemos visto son monoalfabéticos (una letra siempre se cambia por otra de forma fija). Este cifrador polialfabético soluciona esa debilidad (que una letra se cifre siempre igual facilita romperlo).

Si clave + de 6 letras distintas → se difumina redundancia del lenguaje. Aunque hay técnicas de criptoanálisis muy sencillas (Kasiski, Índice de Coincidencia), que rompen la cifra y la clave con mínimos recursos.

Se usa una clave K de tipo String que se repite para que sea tan larga como el mensaje (deben ser letras del alfabeto). Se hace un césar de cada letra del mensaje usando como clave la posición en el alfabeto de letra de la clave que esté en su misma posición. Ejemplo:

Sea K = **CIFRA** y el mensaje M = HOLA AMIGOS

M = H O L A A M I G O S
K = C I F R A C I F R A sumando mod 27...

C = J W **P** R A Ñ **P** L G S Más de un alfabeto: la letra O se cifra de forma distinta.

Haz el programa **U3EPX3.java** que presente un menú similar al del ejercicio 1 y permita configurar el alfabeto, encriptar y desencriptar. Ejemplos de ejecución:

```
    VIGENERE
    ========

1. Alfabeto.
2. Encriptar.
3. Desencriptar.
4. Salir.

Escoja una opción (1-4):
```

```
Escoja una opción (1-4): 1

Alfabeto actual: ABCDEFGHIJKLMNOPQRSTUVWXY Z
Nuevo (enter para no cambiar):
```

```
Escoja una opción (1-4): 2

Mensaje: HOLA 123 HOLA
Clave (una frase): BAB0
   Revisando... clave BAB
Cifra: IOMB ZIOMA
```

```
Escoja una opción (1-4): 3

Cifra: IOMB ZIOMA
Clave (una frase): BAB
   Revisando... clave BAB
Mensaje: HOLA  HOLA
```